Lab 12 — Hands-on lab: deploy, update, roll back

Goal: deploy the API with two replicas and probes, scale it, move it to version 1.1 with no downtime, try a broken version and watch the rollout stall, roll back, then deliberately break a readiness and a liveness to see their opposite effects.

Prerequisites — Lab 11 completed: k3s installed, `localhost/api:1.0` imported into the cluster, Podman with `api:1.0`.

Files provided (`files/`) - `deployment-api.yaml` — the API Deployment: two replicas, readiness and liveness on `/actuator/health`. - `Dockerfile.v11` — builds `api:1.1` (another default message). - `Dockerfile.bad` — builds `api:bad` (an image that does not start).

Step 0 — Two more images in the cluster

```
mkdir -p ~/labo-docker/12 && cd ~/labo-docker/12 && cp <path-to-the-lab>/files/* .
podman build -q -f Dockerfile.v11 -t api:1.1 . && podman build -q -f Dockerfile.bad -t api:bad
.
podman save localhost/api:1.1 | sudo k3s ctr images import -
podman save localhost/api:bad | sudo k3s ctr images import -
sudo k3s crictl images | grep 'localhost/api'
```

Observe two cached builds (a few seconds), two imports, and three `localhost/api` lines in the cluster: `1.0`, `1.1`, `bad`.

Step 1 — The Deployment

```
kubectl create namespace lab12 ; kubectl config set-context --current --namespace=lab12
kubectl apply -f deployment-api.yaml
kubectl rollout status deployment/api --timeout=60s
kubectl get deployment,replicaset,pods -o wide
```

Observe `Waiting for deployment "api" rollout to finish: 1 of 2 updated replicas are available...` then `successfully rolled out`; three levels of objects: `deployment.apps/api 2/2`, `replicaset.apps/api-6c7bb54fc5 2 2 2`, and two Pods `api-6c7bb54fc5-xxxxx Running` each with its IP.

```
kubectl get pods --show-labels
kubectl get pods -l app=api -o name
kubectl describe deployment api | grep -E 'Replicas:|StrategyType|RollingUpdateStrategy'
```

Observe the labels `app=api,pod-template-hash=6c7bb54fc5` on each Pod, the two Pods filtered by `-l app=api`, and the strategy: `RollingUpdate`, `25% max unavailable, 25% max surge`.

Explanation. You wrote a Deployment; the cluster created a ReplicaSet (named after the template hash) and two Pods, linked by the labels. The `rollout status` was waiting for the readiness probes to pass.

Step 2 — Scaling

```
kubectl scale deployment api --replicas=3
kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl get rs
```

Observe a third Pod with the **same** hash, and the ReplicaSet at `3 3 3`.

Explanation. `scale` changes `spec.replicas`; the ReplicaSet does the rest. No new ReplicaSet: the template has not changed.

Step 3 — Moving to 1.1 with no downtime

```
kubectl set image deployment/api api=localhost/api:1.1
kubectl rollout status deployment/api --timeout=90s
kubectl get rs
kubectl rollout history deployment/api
kubectl exec $(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}') -- wget -
qO- http://localhost:8080/ ; echo
kubectl get events --sort-by=.lastTimestamp | grep ScalingReplicaSet | tail -6 | cut -c1-110
```

Observe the progress `1 out of 3 new replicas have been updated…`, `2 out of 3…`, `1 old replicas are pending termination…`, `successfully rolled out`; two ReplicaSets, the new one at `3 3 3` and the old one at `0 0 0`; two revisions in the history; `{"message":"version 1.1",…}`; and the dance of the Events: new `0 → 1`, `1 → 2` / old `3 → 2`, old `2 → 1` / new `2 → 3`, old `1 → 0`.

Explanation. Never fewer than three ready Pods: a new one is created, awaited ready, then an old one removed. The old ReplicaSet stays at zero: it is revision 1, ready for a rollback.

Step 4 — A broken version

```
kubectl set image deployment/api api=localhost/api:bad
sleep 25 ; kubectl get pods
kubectl rollout status deployment/api --timeout=20s ; echo "exit=$?"
kubectl get rs
kubectl logs $(kubectl get rs --sort-by=.metadata.creationTimestamp -o name | tail -1) --tail=1
```

Observe three `1/1 Running` Pods of 1.1 and **a single** new Pod `0/1 Error` with `RESTARTS 2`; a `rollout status` that waits (`1 out of 3 new replicas have been updated…`) then `error: timed out waiting for the condition`, `exit=1`; three ReplicaSets, the new one at `1 1 0`; and the cause, read in the new ReplicaSet's logs: `demarrage impossible : configuration manquante`.

```
kubectl rollout undo deployment/api
kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl rollout history deployment/api
kubectl get deployment api -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
```

Observe `deployment.apps/api rolled back`, the broken Pod in `Terminating` (or still `Error` until it disappears), the three 1.1 Pods untouched, a history `1, 3, 4` (revision 2 became 4 through the rollback), and the image back to `localhost/api:1.1`.

Explanation. The Deployment never removed a single healthy Pod: `maxUnavailable` of 25 % on three replicas is zero. The service was not touched during the incident, and the rollback took one command.

Step 5 — A failing readiness

```
kubectl patch deployment api --type=json -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","valu
e":8081}]'
sleep 25 ; kubectl get pods
kubectl describe pods -l app=api | grep 'Readiness probe failed' | tail -1 | cut -c1-120
kubectl get deployment api
kubectl rollout status deployment/api --timeout=10s ; echo "exit=$?"
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
```

Observe one new Pod `0/1 Running` (it runs, but nothing listens on `8081`) next to the three old `1/1`; `Readiness probe failed: Get "http://10.42.0.x:8081/actuator/health": dial tcp … connection refused`; `READY 3/3 UP-TO-DATE 1 AVAILABLE 3`; a `rollout status` that times out; then the rollback.

Explanation. A Pod that is never ready gets no traffic and does not allow removing an old one: the rollout stalls harmlessly. That is the behaviour you want when a new version does not start correctly.

Step 6 — A failing liveness

```
kubectl patch deployment api --type=json -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/httpGet/port","value
":8081},
{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/initialDelaySeconds","va
lue":2},
{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/periodSeconds","value":3
}]'
sleep 40 ; kubectl get pods
kubectl describe pods -l app=api | grep -E 'Liveness probe failed|Killing' | tail -2 | cut -c1-
120
```

Observe this time **three** new Pods, all `0/1`, in `CrashLoopBackOff` (or `Running` between two relaunches) with `RESTARTS 2`: the rollout replaced the old ones (the readiness passed), then the liveness killed every new Pod; `Liveness probe failed: … connection refused` and `Container api failed liveness probe, will be restarted`.

```
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
kubectl get pods ; kubectl rollout history deployment/api
```

Observe the three healthy Pods back, the broken ones `Terminating`, and a history that skipped numbers again.

Explanation. The rollout only looks at readiness; a broken liveness acts afterwards, on Pods already counted as good, and takes the whole service down. Readiness: protects the deployment; liveness: to be tuned with care.

Clean-up

```
kubectl delete namespace lab12
kubectl config set-context --current --namespace=default
sudo k3s crictl rmi localhost/api:bad
podman rmi api:bad
```

Observe the namespace deleted with its objects, and the broken image removed from the cluster and from Podman. Keep `api:1.0` and `api:1.1` on both sides: lab 13 deploys the complete chain.

What you must be able to state now

- An `apply` of a Deployment creates a ReplicaSet `api-<hash>` and its Pods, linked by `app=api` and `pod-template-hash`.
- `scale` keeps the ReplicaSet; `set image` creates a new one and moves the replicas one by one: `0 → 1`, `3 → 2`, … never fewer than three ready.
- A broken image = a single `Error` Pod, three old ones untouched, `rollout status` in `timed out`, `logs rs/…` for the cause, `rollout undo` to repair.
- Broken readiness = `0/1 Running`, rollout stalled, service intact; broken liveness = everything in `CrashLoopBackOff`.
- `rollout history` skips numbers after every `undo`: normal.